LOK SEWA LMS

Production Build Playbook

Multi-session Claude Code workflow • React Native + PostgreSQL • Student App + Admin CMS + Backend

Purpose

A practical, checkpointed guide for turning the supplied LMS reference PDF into a real-world, end-to-end Lok Sewa preparation platform without depending on a single AI session.

Source baseline: “Tayari Kakshya - LMS System(1).pdf” (49 pages) supplied with this project.

Document version: 1.0 • Prepared for production-oriented implementation planning

1. How to Use This Playbook

Core operating rule

Do not attempt to build the entire product in one Claude Code prompt. Treat every Claude session as a bounded work session, and treat the repository + state documents as the long-term memory.

- Attach the reference PDF when starting the project so Claude Code can inspect it directly.
- Run Phase 0/1 first: requirements discovery and PDF extraction. Do not allow full implementation until business decisions are confirmed.
- Approve the technology stack before major implementation.
- Work in small, testable milestones. After each milestone, require state updates and a handoff report.
- When rate limits hit, open a new session and use the RESUME prompt from Section 15. Never ask only “Did you complete the task?”
- Use the final audit prompts before calling the system production-ready.

1.1 The Persistent-Memory Model

Artifact	Purpose
CLAUDE.md	Permanent project instructions, architecture rules, coding conventions and non-negotiable behavior.
PROJECT_STATE.md	Current real state: what is complete, partial, blocked, in progress and next.
IMPLEMENTATION_PLAN.md	Master milestone checklist and progress tracker.
SESSION_HANDOFF.md	Last-session summary designed for the next Claude session.
DECISIONS.md	Approved product and technical decisions with rationale.
KNOWN_ISSUES.md	Known bugs, blockers and technical debt.
TEST_STATUS.md	What was tested, results and remaining coverage.
PRODUCTION_READINESS_AUDIT.md	Final implementation-vs-requirements audit.

2. Reference PDF: What It Already Contains

The supplied 49-page PDF is the visual baseline. It should be treated as a functional reference, not a limitation on visual quality. The extracted screens include authentication, home/course discovery, subject/chapter learning, video playback, question/quiz flows, weekly exams, news/current affairs, plans, profile, edit profile, notifications and notices. The document also contains design-system/asset references and repeated versions of some screens.

PDF area	Observed functionality	Implementation implication
Pages 1–5	Sign up, login, Google sign-in, forgot password, reset password, success state.	Full authentication flow + secure password lifecycle.
Pages 6–8	Home, search, categories, popular courses, pricing and ratings.	Dynamic catalog, category and course CMS.
Pages 9–11	Course/subject dashboard; subjects; Notes/Videos/Questions/Quizzes; chapter/topic hierarchy.	Course → Subject → Chapter → Topic → Content model.
Pages 12–15	Article/topic view and video player with duration and Previous/Next.	Content viewer + video progress/resume.
Pages 16–18	Chapter questions and MCQ quiz interface with answer display.	Question bank + explanation + quiz engine.
Pages 19–20	Weekly exam and question flow.	Scheduled exam engine + result calculations.

Pages 21–22	Gorkhapatra and Current Affairs screens.	News/current-affairs CMS.
Page 23	Free vs Premium plan cards.	Subscription/entitlement system.
Pages 24–26	Profile, enrolled course, subscription, edit profile.	Account, enrollment and settings.
Pages 27–28	Notifications and notices.	In-app/push communication.
Pages 29–34	Repeated learner screens, login and design/feature references.	Reconcile repeated screens rather than ignoring them.
Pages 35–39	Brand/icon/component references and visual assets.	Use as visual references; improve the design system.
Pages 40–49	Additional subject/chapter/question/video/course/profile references.	Cross-check every repeated/variant feature before implementation.

Important product adaptation

The PDF's example categories and courses are oriented to Nepal Police, Nepal Army, APF, NID and courses such as Sainik Likhit, Officer Cadet and Prahari Jawan. The final product is intended here as a Lok Sewa LMS, so the academic taxonomy, content and business rules must be confirmed rather than copied blindly from the sample data.

3. Final Product Blueprint

The final ecosystem should have three primary surfaces: a student mobile application, an admin CMS, and a production backend/database. The student app should be a dynamic client of the CMS/backend, not a hard-coded collection of screens.

Surface	Primary responsibility
React Native Student App	Learning, practice, exams, progress, payments, profile, content consumption.
Admin CMS	Create/edit/publish/manage every academic, commercial, communication and content entity.
Backend/API	Authentication, authorization, business logic, payment verification, exams, progress, notifications, analytics, media access control.
PostgreSQL	Canonical source of structured application data, relationships, entitlements, attempts, transactions and audit trails.
Object/video storage	Large media and document storage with secure access patterns.

3.1 Core Academic Hierarchy

Recommended model to review with the owner

Exam Authority → Level/Examination → Service → Group → Post → Paper → Subject → Chapter → Topic → Learning/Practice Content. The exact Lok Sewa taxonomy is a business decision and must be confirmed before hard-coding it.

4. Student Mobile App Feature Checklist

Authentication

- Sign up
- Login
- Google sign-in (if approved)
- Logout
- Forgot password
- Reset password
- Change password
- Email/phone verification if approved
- Secure token/session handling
- Device/session management
- Account deletion

Home

- Personalized greeting
- Global search
- Dynamic categories
- Featured/popular/latest courses
- Continue learning
- Daily challenge
- Current affairs
- Upcoming exams
- Notices
- Progress snapshot
- Promotional banners and CTAs

Courses

- Browse/search/filter/sort
- Course details
- Syllabus
- What you will learn
- Instructor
- Duration/lesson counts
- Ratings/reviews
- Original/sale price
- Validity
- Enroll/purchase
- Continue learning

Learning

- Course → Subject → Chapter → Topic
- Notes
- PDFs
- Videos

- Video resume
- Playback speed
- Captions/transcripts where available
- Previous/next
- Progress tracking
- Offline content where approved

Questions & Practice

- Question bank
- MCQ
- multiple-answer
- true/false
- fill-in-the-blank
- match-the-following
- short/long answer
- numerical/image questions
- Filters
- Explanations
- Sources
- Bookmarks
- Report question

Assessments

- Chapter/topic quizzes
- Subject tests
- Daily challenge
- Weekly exams
- Full mock tests
- Previous-year mock mode
- Timers
- Question palette
- Mark for review
- Auto-save
- Auto-submit
- Results and solutions

Current Affairs & News

- Daily/weekly/monthly/yearly current affairs
- Categories
- Quiz
- News/articles
- Gorkhapatra-related section if approved
- Search
- Source/date metadata

Progress & Personalization

- Overall progress

- Subject/topic performance
- Score history
- Weak areas
- Study time
- Study plan
- Bookmarks
- Mistake notebook
- Revision workflows

Communication

- Notifications
- Announcements
- Notices
- Exam reminders
- Course updates
- Purchase/subscription events
- Notification preferences

Profile & Support

- Edit profile
- Enrolled courses
- Subscription
- Settings
- About
- Terms
- Privacy
- Refund policy
- Blog
- FAQ/help
- Support tickets

5. Admin CMS — Complete Dynamic Control Center

The CMS is not a secondary dashboard. It is the control plane for the entire LMS. Every important user-facing data entity should be manageable from the CMS without a code change, subject to deliberate technical constraints.

CMS module	Minimum responsibilities
Dashboard & analytics	KPIs, active users, course enrollments, completion, exam attempts, revenue, subscriptions, difficult questions, system alerts.
Users	Students, instructors, admins, verification, status, enrollments, subscriptions, activity, progress, device/session history.
Roles & permissions	Super Admin, Admin, Content Manager, Question Manager, Instructor, Finance Manager, Support Agent, Editor; granular permissions.
Categories	CRUD, images/icons, descriptions, slugs, order, status.
Exam structure	Authority, level, service, group, post, paper, syllabus, status and ordering.

Courses	Full CRUD, descriptions, thumbnails, pricing, discount, validity, instructor, language, featured/popular, draft/scheduled/published/archived.
Course structure	Subjects, chapters, topics, lessons, drag/reorder, move, duplicate, publish/unpublish.
Notes	Rich text, PDF, images, attachments, access level, order, scheduling, preview, versioning.
Videos	Upload, thumbnails, durations, captions, transcripts, access control, order, publish/schedule, replace.
Question bank	Question types, options, answers, explanations, marks, negative marks, difficulty, tags, exam/post/paper/year, sources, images.
Question import	CSV/XLSX template, validation, preview, duplicate detection, import summary, bulk operations.
Question reports	Review, assign, resolve, reject, fix, audit trail.
Quizzes	Builder, random/manual selection, timer, marks, negative marking, pass mark, attempts, access.
Mock exams	Blueprints, subject distribution, duration, scoring, randomization, scheduling, attempt rules.
Previous-year questions	Year/exam/post/paper/subject, answers, solutions, explanations and source.
Current affairs	Daily/weekly/monthly/yearly, categories, source, tags, featured, scheduling, quizzes.
News/blog	Article CRUD, categories, tags, source, author, images, SEO, scheduling.
Notices & notifications	Audience selection, attachments, priority, scheduling, deep links, read/unread.
Subscriptions	Plans, features, duration, included courses/content/exams, status.
Pricing & coupons	Original/sale price, promotions, validity, coupons, limits and eligibility.
Payments/refunds	Transactions, gateway references, success/pending/failed/cancelled/refunded/reconciled, refund workflow.
Invoices/receipts	Invoice ID, customer, item, amount, provider reference, status, date.
Home/app CMS	Banners, sections, CTAs, featured content, ordering, visibility, target audience.
Media library	Images, videos, PDFs, documents, reuse, metadata, replace/delete.
FAQ/static pages	About, Terms, Privacy, Refund, Contact, Help, FAQ.
Progress/results	Student progress, attempts, scores, time, subject performance, exports.
Support	Tickets, priority, assignment, status, attachments, conversation.
Audit & system	Audit logs, settings, feature flags, backups, security controls.

6. Dynamic CMS Rules

- Adding a course must not require a mobile app release.
- Changing a course price must not require a code change.
- Adding subjects/chapters/topics/questions/videos/notes must be possible from the CMS.
- Home banners, sections and CTAs should be CMS-controlled where practical.
- Premium/free access should be enforced server-side.
- Question/exam scoring rules should be data-driven and validated on the backend.

- Important financial/audit records should not be destructively deleted.

7. Technology Decision Gate

Preferred baseline: React Native + PostgreSQL. Claude Code must still present alternatives and wait for approval before locking major architecture choices.

Area	Options to compare	Decision questions
Mobile setup	Expo vs React Native CLI/Bare	Native modules, build workflow, payment SDK compatibility, OTA updates, maintenance.
Backend	NestJS, Fastify, Express, Next.js API architecture	Architecture, scalability, validation, background jobs, maintainability.
API	REST, GraphQL, tRPC	Mobile fit, versioning, caching, complexity and maintainability.
ORM	Prisma, Drizzle, TypeORM, raw SQL mix	Migrations, query control, productivity, team familiarity.
Auth	JWT + refresh, sessions, managed auth	Security, mobile storage, social login, OTP, cost.
State	Redux Toolkit, Zustand, Context	Complexity, predictability, performance.
Storage	S3, Cloudflare R2, managed storage	Cost, CDN, egress, security, scalability.
Video	Mux, Cloudflare Stream, Bunny Stream, storage + CDN	Transcoding, adaptive bitrate, security, cost, playback.
Cache	Redis / managed Redis / PostgreSQL only	Hot data, rate limits, jobs, cost.
Search	PostgreSQL FTS, Meilisearch, OpenSearch/Elasticsearch	Scale, relevance, operational burden.
Jobs	BullMQ/Redis, cloud queue, other	Reliability, retries, scheduling, operational cost.
Notifications	FCM/APNs, Expo notifications if applicable	Platform coverage, reliability, token lifecycle.
Admin	Next.js, React + Vite, other	CRUD ergonomics, routing, auth, deployment.

Decision rule

Claude should recommend a stack based on the actual product constraints and show pros/cons. The user approves the final selection before major implementation.

8. Credentials & Environment Intake

The app is intended to be production-ready, so real service credentials must never be fabricated. Use placeholders or sandbox credentials during development and ask for real production credentials only when the relevant integration is ready.

Area	Typical inputs Claude should request	Secret?
PostgreSQL	DATABASE_URL, database host/user/password/database/SSL as applicable	Yes
Backend	APP_URL, API URL, CORS origins, auth secrets	Yes
Google OAuth	Client IDs, client secret/config as applicable	Yes/partly
Email	SMTP/provider API key, sender address	Yes
OTP/SMS	Provider credentials, sender ID, templates	Yes
eSewa	Sandbox/production merchant identifiers	Yes

	and provider configuration	
Khalti	Sandbox/production API credentials and provider configuration	Yes
ConnectIPS	Merchant/institution configuration and credentials	Yes
Storage	Bucket/account credentials, region, endpoint	Yes
Video	Video provider API credentials	Yes
Push	Firebase/APNs credentials/config	Yes/partly
Analytics/monitoring	Project keys / DSN / app identifiers	Often yes
Search/cache/jobs	Managed service URLs/credentials if used	Yes
Deployment	Hosting/project IDs, registry credentials, CI/CD secrets	Yes

Never do this

Never paste secrets into source code, never commit .env files containing credentials, and never treat a client-side payment success response as proof of payment.

9. Full Build Roadmap

Phase	Name	Outcome
0	Requirements discovery	Collect missing business/product decisions; inspect the PDF; define scope boundaries.
1	PDF extraction	Page-by-page screen and feature inventory; visual reconciliation.
2	Suggested features	List missing LMS capabilities; user approves what becomes product scope.
3	Technology selection	Compare stack alternatives; user approves final stack.
4	Architecture	Repository layout, service boundaries, API approach, data ownership.
5	Database	ERD, migrations, indexes, seed strategy, audit model.
6	Backend foundation	Config, auth foundation, validation, error handling, logging, RBAC.
7	Admin CMS foundation	Layout, auth, RBAC, CRUD framework, tables/forms, media foundation.
8	Academic/content CMS	Categories, exam hierarchy, courses, subjects, chapters, topics, notes, videos.
9	Question/assessment CMS	Questions, import, quizzes, previous-year, mock exams, weekly exams.
10	Student mobile foundation	Navigation, design system, auth, API client, state, caching, error/loading states.
11	Student learning	Course catalog, course details, notes, videos, questions, quizzes.
12	Exam engine	Mock/weekly exams, timers, save/resume, scoring, results, analytics.
13	Commerce	Plans, checkout, eSewa, Khalti, ConnectIPS, orders, entitlements, receipts.
14	Communication/content	Current affairs, news, notices, notifications, blog.
15	Progress/personalization	Progress, analytics, study plan,

		bookmarks, mistake notebook, revision.
16	Advanced approved features	Offline, discussion, AI assistant, gamification, etc. only if approved.
17	Hardening	Security, performance, reliability, accessibility, observability.
18	Testing	Unit, integration, E2E, payment edge cases, authorization tests.
19	Production audit	Compare implementation against requirements and PDF; fix gaps.
20	Release	Deployment, monitoring, store readiness, documentation, backup/recovery.

10. MASTER PROMPT TO START THE PROJECT

Copy the following prompt into the first Claude Code session and attach the 49-page PDF. It is intentionally long because it establishes the rules for all later sessions.

Copy/paste master prompt

You are the Principal Software Architect, Senior React Native Engineer, Senior Backend Engineer, Database Architect, UI/UX Designer, Admin CMS Engineer, QA Engineer, DevOps Engineer, Security Engineer and Technical Lead for a production Lok Sewa Learning Management System.

This is a REAL product, not a college/demo project. Build a complete ecosystem: React Native student mobile app + web Admin CMS + backend/API + PostgreSQL + production integrations. The system must be maintainable, secure, testable, scalable and deployable.

HARD RULE: DO NOT START FULL IMPLEMENTATION IMMEDIATELY.
First inspect requirements, inspect the attached PDF page-by-page, extract its features, identify missing inputs, compare technology alternatives, and ask me for the decisions that must be confirmed.

PDF RULE:
- Extract ALL pages of the attached PDF very carefully.
- Inspect both parsed text and visual screenshots.
- Do not skip repeated/variant pages.
- Build PDF_UI_FEATURE_EXTRACTION.md with page, screen, UI, actions, navigation, data dependencies, backend implications and ambiguities.
- The PDF is a reference, not a limitation. Preserve functionality but create a much more modern, aesthetic, premium and professional design if the reference UI is weak.

DESIGN:
- Modern, premium, elegant, clean, mobile-first.
- Better typography, spacing, hierarchy, accessibility, animations and visual polish than the reference where appropriate.
- Avoid excessive gradients, clutter, random colors and template-looking UI.
- Build a reusable design system.

TECHNOLOGY:
- My preferred mobile technology is React Native.
- PostgreSQL is the canonical primary database.
- Before locking other major technologies, compare suitable alternatives with pros/cons, cost, scalability, maintenance and integration implications, then ask me to approve.
- Compare at minimum mobile setup, backend framework, API style, ORM, auth, state management, storage, video delivery, cache, search, queues/jobs, push notifications, and Admin CMS stack.

CREDENTIALS:
- Create ENVIRONMENT_REQUIREMENTS.md and .env.example.
- Ask me for any credentials required for real services.
- Never fabricate production secrets.
- Use sandbox/test credentials until production credentials are explicitly provided.
- Never commit secrets.

PAYMENTS:
The product must support course/subscription purchases using eSewa, Khalti and ConnectIPS, subject to current official integration requirements.
- Verify the latest official provider documentation before implementation.
- Use a payment-provider abstraction.
- Implement server-side payment verification.
- Handle pending, success, failure, cancelled, expired, refunded and reconciled states as supported.
- Prevent duplicate callbacks, amount tampering, replay and unauthorized enrollment.

DYNAMIC CMS:
The mobile app must not hard-code academic or commercial content. Admins must be able to manage categories, exam/post hierarchy, courses, pricing, subscriptions, subjects, chapters, topics, notes, videos, questions, quizzes, mock exams, previous-year questions, current affairs, news, notices, notifications, banners, FAQs, blog/static pages and other approved content from the CMS without developer intervention.

STUDENT APP:
Support the approved scope for authentication, home, course catalog, course details, my learning, subject/chapter/topic learning, notes, PDFs, videos, questions, quizzes, mock exams, weekly exams, previous-year questions, current affairs, news, notices, notifications, subscriptions, payments, progress, analytics, study plans, bookmarks, mistake notebook, revision, profile and support.

ADMIN CMS:
Support complete CRUD, search/filter/sort/pagination, bulk operations where useful, import/export, draft/review/publish workflows, scheduling, soft deletion/restore, audit logs, RBAC, analytics and system settings across the approved modules.

ACADEMIC MODEL:
Propose and validate an extensible Lok Sewa hierarchy such as Exam Authority -> Level/Examination -> Service -> Group -> Post -> Paper -> Subject -> Chapter -> Topic -> Content. Do not assume exact taxonomy or business rules; ask me where the decision is mine.

SECURITY:
Use server-side authorization, RBAC, secure credential storage, rate limiting, validation, safe file handling, audit logs, payment verification, secure token handling and appropriate protections against common web/API/mobile security issues.

TESTING:
Critical flows require automated tests: authentication, course access, payments, enrollment, quizzes, exam scoring, exam submission, subscription expiry, CRUD, RBAC and important API behavior.

PERSISTENT PROJECT MEMORY:
Create and maintain:
- CLAUDE.md
- docs/PROJECT_STATE.md
- docs/IMPLEMENTATION_PLAN.md
- docs/DECISIONS.md
- docs/KNOWN_ISSUES.md
- docs/ENVIRONMENT_STATUS.md
- docs/TEST_STATUS.md

- docs/CHANGELOG.md
- docs/SESSION_HANDOFF.md

At the beginning of every new session, read those files, inspect the repository and git status, and reconcile docs with actual code. Never rely on conversation memory.

STOP AFTER DISCOVERY:
In your FIRST response, do not build the entire app. Inspect the PDF, extract features, identify missing requirements, compare stack alternatives, create the documentation above, and ask me a grouped set of questions before major implementation.

11. Phase-by-Phase Claude Code Prompts

11.1 Phase 0 — Requirements + PDF extraction

Start Phase 0 and Phase 1 only.

Inspect the attached PDF carefully, page-by-page, including visual layouts and repeated/variant screens.
Do not start the full application implementation.

Create/update:
- PDF_UI_FEATURE_EXTRACTION.md
- FEATURE_SPECIFICATION.md
- SUGGESTED_FEATURES.md
- PROJECT_STATE.md
- IMPLEMENTATION_PLAN.md
- DECISIONS.md
- ENVIRONMENT_REQUIREMENTS.md
- CLAUDE.md

For every PDF screen record page number, screen name, UI components, user actions, navigation, data dependencies, implied backend needs and ambiguities.

Identify missing product/business inputs but do not silently invent them.
Separate:
1. PDF-derived requirements
2. Explicit user requirements
3. Recommended features awaiting approval
4. Technical implementation decisions

At the end, ask me the minimum complete grouped set of questions required to proceed.
Stop before full implementation.

11.2 Phase 2 — Technology alternatives and approval

Resume after I have answered the requirements questions.

First read all project-state and decisions documents.

Create TECH_STACK_OPTIONS.md with a serious comparison for:
- React Native setup
- Backend framework
- API style
- PostgreSQL ORM

- Authentication
- State management
- File storage
- Video delivery
- Cache
- Search
- Background jobs
- Notifications
- Admin CMS
- Monitoring/analytics

For each option show pros, cons, cost/operational implications, scalability and fit for this LMS.

Do not lock the final stack until I approve it.
Once approved, record the decision in DECISIONS.md.

11.3 Phase 3 — Architecture + repository structure

Resume the project.

Read CLAUDE.md, PROJECT_STATE.md, IMPLEMENTATION_PLAN.md, DECISIONS.md and the PDF feature extraction.

Design the complete system architecture:
- mobile app
- admin CMS
- backend/API
- PostgreSQL
- storage/video
- payments
- notifications
- search/cache/jobs where approved
- monitoring/logging

Create/update:
- ARCHITECTURE.md
- DATABASE_ARCHITECTURE.md
- API_DOCUMENTATION.md
- repository structure documentation
- PROJECT_STATE.md

Do not begin broad feature implementation until the architecture is internally consistent.
Show unresolved product decisions separately.

11.4 Phase 4 — PostgreSQL + backend foundation

Resume the project from the actual repository state.

Implement only the approved backend foundation:
- project setup
- configuration
- PostgreSQL connection
- migrations
- ORM/query layer
- validation
- error handling

- logging
- authentication foundation
- authorization foundation
- RBAC
- rate limiting/security foundation
- API structure
- health checks

Do not invent product rules.
Use real persistence; do not use permanent mock APIs.
Run tests for the foundation.
Update project-state and handoff documents before stopping.

11.5 Phase 5 — Admin CMS foundation

Resume the project.

Build the web Admin CMS foundation using the approved stack.
Implement:
- admin authentication
- RBAC
- dashboard shell
- navigation
- reusable data table
- reusable form system
- validation
- loading/empty/error states
- search/filter/sort/pagination patterns
- audit-aware actions
- media foundation

Keep the CMS connected to the real backend.
Do not replace backend behavior with permanent client-side mock data.

11.6 Phase 6 — Academic/content CMS

Continue from the current repository state.

Implement the approved academic/content modules:
- categories
- exam authority/level/service/group/post/paper hierarchy
- courses
- course pricing
- subjects
- chapters
- topics
- notes
- PDFs
- videos
- media library

Everything must support the appropriate CRUD workflow and persist to PostgreSQL.
Add ordering/reordering, publish states, scheduling and soft deletion where approved.
Verify that a newly created item can be retrieved by the student-facing API without code changes.

11.7 Phase 7 — Question bank + assessments CMS

Continue from the current repository state.

Implement:
- question types
- question CRUD
- explanations
- marks and negative marks
- difficulty/tags/source metadata
- image-based questions where approved
- question reports
- question versioning
- CSV/XLSX import with validation/preview
- quiz builder
- previous-year question management
- mock exam builder
- weekly exam management

Use real persistence and backend validation.
Create automated tests for question evaluation and exam scoring.

11.8 Phase 8 — Student mobile foundation

Resume the project.

Implement the React Native foundation using the approved stack and design system.

Build:
- navigation
- theme/design tokens
- typography
- reusable buttons/inputs/cards/tabs/modals
- API client
- auth state
- secure token storage
- loading/empty/error states
- network handling
- local caching where approved

Match the approved product behavior from the PDF extraction while significantly improving visual quality.
Do not hard-code academic content that belongs in the CMS/database.

11.9 Phase 9 — Student learning experience

Resume from the actual repository state.

Implement and connect:
- Home
- Categories
- Course catalog
- Course details
- Enrollment state
- My Learning
- Subjects

- Chapters
- Topics
- Notes/PDFs
- Videos
- Progress/resume position
- Questions
- Quizzes
- Bookmarks
- Report question

Every screen must support correct loading/empty/error/success states and real API data.

11.10 Phase 10 — Exams + results

Resume the project.

Implement the approved exam experience:
- daily challenge if approved
- chapter/subject practice
- weekly exam
- full mock exam
- previous-year mock mode
- question palette
- timer
- save answers
- mark for review
- submit confirmation
- auto-submit on time expiry
- server-validated scoring
- result breakdown
- explanations/solutions
- attempt history

Test edge cases such as reconnects, repeated submit, expired timer and duplicate attempt requests.

11.11 Phase 11 — Payments: eSewa, Khalti, ConnectIPS

Resume only after the payment architecture and business rules are approved.

Before coding provider-specific behavior, verify the latest official integration requirements for eSewa, Khalti and ConnectIPS.

Implement a provider abstraction and the approved flow:
Course/Plan -> Checkout -> Payment -> Provider callback/webhook -> Server-side verification -> Order state -> Enrollment/Subscription -> Receipt -> Notification.

Use sandbox/test credentials unless I explicitly provide production credentials.

Test:
- success
- failure
- cancellation
- duplicate callback
- invalid signature
- invalid amount
- expired payment

- client disconnect after payment
- repeated callback
- duplicate enrollment

Never trust client-side success alone.

11.12 Phase 12 — Current affairs, notices and notifications

Resume the project.

Implement approved communication/content modules:
- current affairs daily/weekly/monthly/yearly
- news/articles/Gorkhapatra-related content if approved
- notices
- in-app notifications
- push notifications
- notification preferences
- deep links
- scheduling

Everything should be CMS-driven.

11.13 Phase 13 — Progress, analytics and personalization

Resume the project.

Implement the approved learner analytics:
- course progress
- subject/chapter/topic progress
- study time
- quiz history
- mock history
- score history
- accuracy
- weak areas
- strong areas
- bookmarks
- mistake notebook
- revision workflows
- study plan if approved

Ensure calculations are consistent between backend and mobile UI.

11.14 Phase 14 — Advanced approved features only

Resume the project.

Implement only advanced features that I have explicitly approved:
- offline learning
- discussion/doubt system
- AI study assistant
- gamification
- leaderboard
- certificates

Before implementing an advanced feature, confirm the relevant decision in DECISIONS.md.
Do not invent business rules.

11.15 Phase 15 — Security hardening

Resume the project.

Perform a security hardening pass.
Inspect:
- authentication
- authorization
- RBAC
- IDOR/BOLA risks
- input validation
- file uploads
- payment verification
- rate limits
- token storage
- logging of secrets
- admin privilege escalation
- premium access enforcement
- API error leakage
- database safety
- audit logs

Fix verified issues and document remaining risks.

11.16 Phase 16 — Full testing and bug fixing

Resume the project.

Do not add new product features in this session.

Run a systematic test pass across:
- authentication
- course browsing
- enrollment
- payments
- premium access
- notes
- videos
- questions
- quizzes
- mock exams
- weekly exams
- scoring
- results
- progress
- notifications
- current affairs
- Admin CRUD
- RBAC
- security

Fix actual defects, rerun tests and update TEST_STATUS.md, PROJECT_STATE.md and KNOWN_ISSUES.md.

12. What to Do When Claude Code Rate-Limits or Stops

Do not ask “Did you complete the task?”

A vague completion question can cause a new session to over-trust the prior conversation. Use the repository state as the source of truth instead.

12.1 Standard resume prompt

```
RESUME THE PROJECT FROM THE CURRENT REPOSITORY STATE.

This is a continuation of a previous Claude Code session.
Do NOT assume the previous session completed its work.

Before doing anything:
1. Read CLAUDE.md.
2. Read docs/PROJECT_STATE.md.
3. Read docs/IMPLEMENTATION_PLAN.md.
4. Read docs/SESSION_HANDOFF.md.
5. Read docs/DECISIONS.md.
6. Read docs/KNOWN_ISSUES.md.
7. Read docs/TEST_STATUS.md.
8. Inspect the repository structure.
9. Run git status.
10. Inspect recent git history.
11. Inspect the actual implementation and relevant tests.

Reconcile documentation with the actual codebase.
Do not restart or duplicate completed work.

Before modifying code, briefly state:
CURRENT PHASE:
CURRENT TASK:
ACTUALLY COMPLETED:
IN PROGRESS:
REMAINING:
NEXT WORK:

Then continue the exact unfinished task.

When you reach a safe stopping point:
- update PROJECT_STATE.md
- update IMPLEMENTATION_PLAN.md
- update SESSION_HANDOFF.md
- update TEST_STATUS.md if tests changed
- record blockers/known issues
- never claim completion without verification
```

12.2 Resume a specific unfinished module

RESUME THE PROJECT.

The last session may have stopped due to context/rate limits.

Read all persistent project-state documents and inspect the codebase.
The module to continue is: [MODULE NAME]

Determine exactly what is already implemented, partially implemented and missing.
Do not recreate existing work.
Do not infer completion from conversation history.

Continue from the precise unfinished implementation point.
Run the relevant tests and update project-state documentation before stopping.

13. Session Handoff / End-of-Session Prompt

13.1 Ask Claude to leave the repository resumable

Before ending this session, prepare a clean handoff for the next Claude Code session.

Do not claim that incomplete work is complete.

Update:
- docs/PROJECT_STATE.md
- docs/IMPLEMENTATION_PLAN.md
- docs/SESSION_HANDOFF.md
- docs/KNOWN_ISSUES.md
- docs/TEST_STATUS.md
- docs/CHANGELOG.md

SESSION HANDOFF must contain:
- current phase
- current task
- completed items
- in-progress items
- unfinished items
- exact next steps
- files changed
- database changes
- API changes
- tests run and results
- known issues
- blockers
- credentials still required
- user decisions still required

If a task is half implemented, clearly identify the partial state and the exact file/module to continue.

14. Production Audit Prompts

14.1 Full production readiness audit

Perform a FULL PRODUCTION READINESS AUDIT of the current project.

Do not modify the application immediately.
First inspect the repository and compare it against:
1. approved requirements
2. PDF_UI_FEATURE_EXTRACTION.md
3. FEATURE_SPECIFICATION.md
4. IMPLEMENTATION_PLAN.md
5. DATABASE_ARCHITECTURE.md
6. API_DOCUMENTATION.md
7. ADMIN CMS requirements
8. MOBILE APP requirements
9. payment/security requirements

Create docs/PRODUCTION_READINESS_AUDIT.md.

For every feature report:
FEATURE:
STATUS: COMPLETE / PARTIAL / MISSING / BROKEN / BLOCKED
EVIDENCE:
Files/endpoints/screens/tables/tests that support the status.

Also identify:
- missing features
- incomplete CRUD
- hard-coded production content
- mock APIs
- fake payment verification
- security gaps
- database issues
- API issues
- mobile UX issues
- admin CMS issues
- performance issues
- missing tests
- missing environment variables
- deployment gaps

Do not claim production readiness without evidence.
Stop after the audit.

14.2 Fix the approved audit findings

Implement the approved fixes from docs/PRODUCTION_READINESS_AUDIT.md.

Work systematically by finding category.
For each finding:
- reproduce or confirm the issue
- fix it
- test the fix
- update the audit status

- update project state
- update known issues

Do not mark a finding resolved without verification.
Do not introduce unrelated new features.

14.3 Final release audit

Perform the final release audit.

Verify:
- build configuration
- environment variable coverage
- no committed secrets
- database migrations
- backups/recovery documentation
- authentication
- authorization/RBAC
- payments
- course entitlements
- exam engine
- scoring
- notifications
- analytics/monitoring
- crash/error handling
- tests
- mobile release identifiers/signing placeholders
- admin deployment
- backend deployment
- API documentation
- privacy/terms/refund requirements supplied by owner

Generate RELEASE_READINESS.md with evidence and remaining blockers.
Do not call the system production-ready if critical blockers remain.

15. Backend & Database Non-Negotiables

- PostgreSQL is the canonical relational data store unless the approved architecture explicitly adds other stores for specialized workloads.
- Use migrations, constraints, foreign keys and indexes based on real query patterns.
- Model domain entities and relationships rather than building one table per screen.
- Server-side authorization is mandatory; mobile-side checks are not security boundaries.
- Course prices, plan access, exam rules and payment states must be validated on the backend.
- Large lists must be paginated; large question banks must not be loaded wholesale into mobile memory.
- Use idempotency for payment callbacks and other naturally repeatable external events.
- Use audit trails for sensitive admin changes and financial operations.
- Use soft delete/restore where business value warrants recovery; protect audit/financial records from destructive deletion.

15.1 Suggested Domain Model

Domain	Representative entities

Identity	Users, Roles, Permissions, Sessions, Verification, Auth events
Academic	Authorities, Levels, Services, Groups, Posts, Papers, Syllabus
Learning	Courses, Enrollments, Subjects, Chapters, Topics, Lessons, Notes, Videos, Media
Assessment	Questions, Options, Tags, Reports, Versions, Quizzes, Exams, Attempts, Answers
Content	Current Affairs, News, Blog Posts, Notices, Banners, FAQs, Static Pages
Commerce	Plans, Subscriptions, Orders, Payments, Refunds, Coupons, Invoices
Engagement	Bookmarks, User Notes, Mistakes, Progress, Study Plans, Reviews
Operations	Notifications, Tickets, Audit Logs, Settings, Feature Flags, Jobs

16. End-to-End User Flows

Flow	Steps
New student	Open app → Sign up → Verify if required → Login → Choose target exam/post → Browse content → Purchase or enroll → Access course → Learn → Practice → Take mock → Review result.
Returning student	Open app → Secure session restore → Home → Continue learning → Study → Practice → Progress saved automatically.
Course purchase	Course detail → Checkout → Payment method → Provider flow → Backend verification → Order success → Entitlement activation → Receipt → Notification.
Quiz	Open quiz → Instructions → Timer (if applicable) → Answer questions → Mark/review → Submit → Server scoring → Result → Explanation → Attempt history.
Mock exam	Open scheduled exam → Eligibility check → Start → Timer → Auto-save → Submit/auto-submit → Score → Detailed analysis → History.
Admin content	Admin login → RBAC check → Create/edit entity → Validate → Save draft → Review → Publish/schedule → API exposes content → Mobile renders it.
Reported question	Student reports → CMS queue → Admin/Question Manager review → Fix or reject → Resolution logged → Updated content served.
Payment failure recovery	Checkout → Provider attempt → Failure/cancelled/pending → Order remains non-entitled → Retry or support flow → Verified success activates entitlement.

17. Design & UX Quality Gate

- Use the PDF for information architecture and feature reference, not as a command to preserve weak styling.
- Create a coherent brand system with accessible color contrast, typography hierarchy, spacing and reusable components.
- Use animation to improve comprehension and feedback, never to distract from learning or exams.
- Provide loading, empty, error and success states for every important screen.
- Design for poor connectivity and mobile constraints common to real users.
- Use Nepali Unicode correctly and plan for Nepali + English if approved.

- Keep the exam interface fast, low-distraction and reliable.

18. Definition of Done

Area	Done when…
Requirements	All critical product decisions are confirmed and documented.
PDF coverage	Every meaningful PDF feature is mapped and reconciled.
Mobile	Approved user journeys work end-to-end against real APIs.
Admin	Approved entities support complete CRUD and publishing workflows.
Backend	Validation, authorization, errors, logging and persistence are tested.
Database	Migrations, constraints and indexes are in place.
Payments	Provider flow is verified server-side in sandbox/test and production configuration is documented.
Exams	Timer, submission, scoring, retry/edge cases and results are tested.
Security	Critical authorization/payment/file/API risks are addressed.
Testing	Critical flows have automated coverage and pass.
Operations	Environment variables, monitoring, backup and deployment are documented.
Documentation	README, architecture, API, deployment, security and handoff docs are current.

19. Common Claude Code Session Problems

Problem	Response
Rate limit / context ends	Start new session with Standard Resume Prompt; let Claude inspect repository state.
Claude claims completion too early	Run Production Readiness Audit instead of accepting the claim.
Claude starts redoing work	Tell it to reconcile docs with git/code and not duplicate implementation.
Mocks remain in production code	Run audit for hard-coded/mock behavior and replace with real persistence/API.
Payment logic is client-trusted	Stop and require server-side verification and idempotency.
Too many new features appear	Return to SUGGESTED_FEATURES.md and DECISIONS.md; implement only approved features.
Requirements missing	Do not invent them; ask grouped questions and record decisions.
UI looks like generic template	Run a design-system/UX review and reconcile against the PDF's intent plus approved brand direction.

20. Quick Copy-Paste Prompts

20.1 "Continue from here"

Continue from the actual repository state.
Read CLAUDE.md, PROJECT_STATE.md, IMPLEMENTATION_PLAN.md, SESSION_HANDOFF.md and DECISIONS.md.
Inspect git status and the relevant code/tests.
Tell me the exact unfinished task, then continue without duplicating completed work.

Update project-state files before stopping.

20.2 “Audit before coding”

Before writing code, inspect the existing implementation of [FEATURE].
Determine what exists, what is partial, what is broken and what is missing.
Show evidence from the repository.
Only then propose the smallest implementation plan needed.

20.3 “No assumptions”

Do not invent product/business rules.
Separate confirmed requirements from recommendations and technical defaults.
Where a product decision is missing, ask me and wait.

20.4 “Verify before claiming done”

Do not claim this feature is complete based only on file existence or compilation.
Run the relevant tests and verify the end-to-end behavior.
Report evidence, limitations and remaining work.

21. Owner Approval Checklist

Approval item	Confirmed?
App name/brand identity confirmed	[ ]
Logo/assets confirmed	[ ]
Target Lok Sewa scope confirmed	[ ]
Target posts/services/groups confirmed	[ ]
Academic hierarchy confirmed	[ ]
Course model confirmed	[ ]
Course pricing confirmed	[ ]
Subscription model confirmed	[ ]
Course validity confirmed	[ ]
Discount/coupon rules confirmed	[ ]
Question types confirmed	[ ]
Negative-marking rules confirmed	[ ]
Exam duration/pass-mark rules confirmed	[ ]
Mock/weekly exam rules confirmed	[ ]
Previous-year content source confirmed	[ ]
Current affairs publishing model confirmed	[ ]
Notification policy confirmed	[ ]
Language requirements confirmed	[ ]
Offline learning decision confirmed	[ ]
AI assistant decision confirmed	[ ]
Discussion/support decision confirmed	[ ]
Payment gateway account ownership/configuration confirmed	[ ]
Refund policy confirmed	[ ]
Invoice/tax requirements confirmed	[ ]
Support contact confirmed	[ ]
Production domain/hosting confirmed	[ ]
Analytics/monitoring confirmed	[ ]
App Store / Play Store ownership confirmed	[ ]
Privacy/Terms/Refund content supplied	[ ]

Production credential strategy confirmed	[ ]

Appendix A — Recommended CLAUDE.md Content

A.1 Permanent project rules

This repository is the source of truth for a production Lok Sewa LMS.

Permanent rules:
1. Never fabricate secrets or production credentials.
2. Never silently invent product requirements.
3. Read docs/PROJECT_STATE.md, docs/IMPLEMENTATION_PLAN.md, docs/DECISIONS.md and docs/SESSION_HANDOFF.md before starting a new session.
4. Reconcile docs with actual repository code and git status.
5. Do not duplicate completed work.
6. Treat PostgreSQL as the canonical primary relational database unless the approved architecture states otherwise.
7. Keep student app, Admin CMS and backend concerns separated.
8. Enforce authorization on the server.
9. Never trust client-side pricing or payment success.
10. Do not call features complete without verification.
11. Update state/handoff documentation after meaningful implementation work.
12. Follow the approved architecture and decisions unless explicitly changed by the owner.

Appendix B — Suggested Project State Template

B.1 PROJECT_STATE.md template

```
# Project State

Current Phase:
Current Milestone:
Overall Status: NOT STARTED / IN PROGRESS / PARTIAL / BLOCKED / RELEASE-READY

## Completed
-

## In Progress
-

## Remaining
-

## Files Changed
-

## Database Changes
-

## API Changes
-
```

Mobile Changes
-

Admin CMS Changes
-

Tests
-

Known Issues
-

Blockers
-

User Decisions Required
-

Credentials Required
-

Exact Next Task
-